



SmartCart

A Smarter Way to Shop

Presented by:
Milla Kovshov, Maria Ishak, Austin Mason



Table of contents

01

**Our
Concept**

02

**Functional
Requirements**

03

**Non-Functional
Requirements**

04

**System
Constraints**

05

Evolution

06

**Use Case &
Diagrams**

07

System Model

08

**System
Architecture**

09

**Design
Patterns**

10

**Mockup
Interfac**

11

Conclusion



01

Our Concept

Maria, Austin, Milla

SmartCart



What is SmartCart?

- Centralized grocery pricing system
- Problem: Shoppers check multiple apps and flyers separately to find the best deals.
- Solution: One platform to search, compare, calculate, and save.
- Target users: Students and budget-conscious grocery shoppers.
- Web-based platform accessible from any standard browser.



Maria, Austin, Milla

SmartCart



02

Functional Requirements



Functional Requirements



FR 1.1.1 — Grocery Item Search

The system allows users to search for grocery items and view pricing from multiple stores. Centralizes data so users find all prices in one place.
Precondition: Pricing data exists.

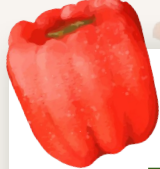
Priority: 0

FR 1.1.2 — Price Comparison

The system compares prices of the same item across all stores and displays a table highlighting the lowest price automatically. Precondition: User has searched an item.

Priority: 0





Functional Requirements (cont.)

FR 1.1.3 — Cart Cost Calculation

The system calculates the total cost of a user's grocery list per store, so users can estimate spending before shopping. Precondition: User has added multiple items.

Priority: 1

FR 1.1.4 — Savings Highlight

The system highlights potential savings by showing the difference between the cheapest and most expensive store options for the full cart.

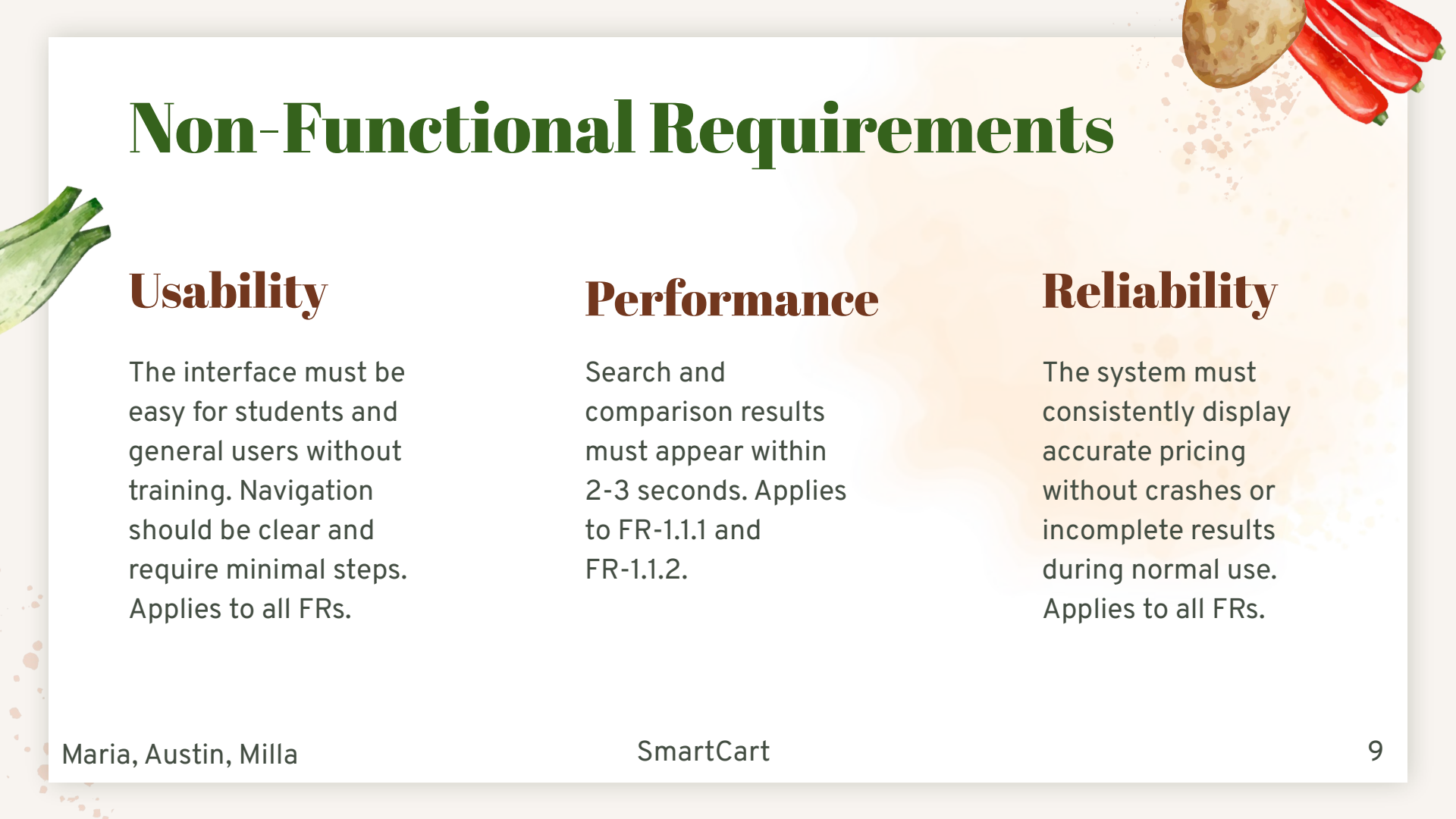
Priority: 2



The slide features a white background with decorative elements. There are four red apples: one at the top left, one at the top right, one at the bottom left, and one at the bottom right. Additionally, there are orange and yellow splatters or watercolor-like textures in the top right and bottom left corners.

03

Non-Functional Requirements



Non-Functional Requirements

Usability

The interface must be easy for students and general users without training. Navigation should be clear and require minimal steps. Applies to all FRs.

Performance

Search and comparison results must appear within 2-3 seconds. Applies to FR-1.1.1 and FR-1.1.2.

Reliability

The system must consistently display accurate pricing without crashes or incomplete results during normal use. Applies to all FRs.

The slide features a white background with decorative elements. In the top-left corner, there is a small red apple. In the top-right corner, there is a larger red apple. In the bottom-left corner, there is a cluster of orange and red splatters. In the bottom-center, there is a red apple. The number '04' is displayed in a large, dark brown serif font within an orange rectangular box.

04

System Constraints

System Constraints

Web-Based Platform

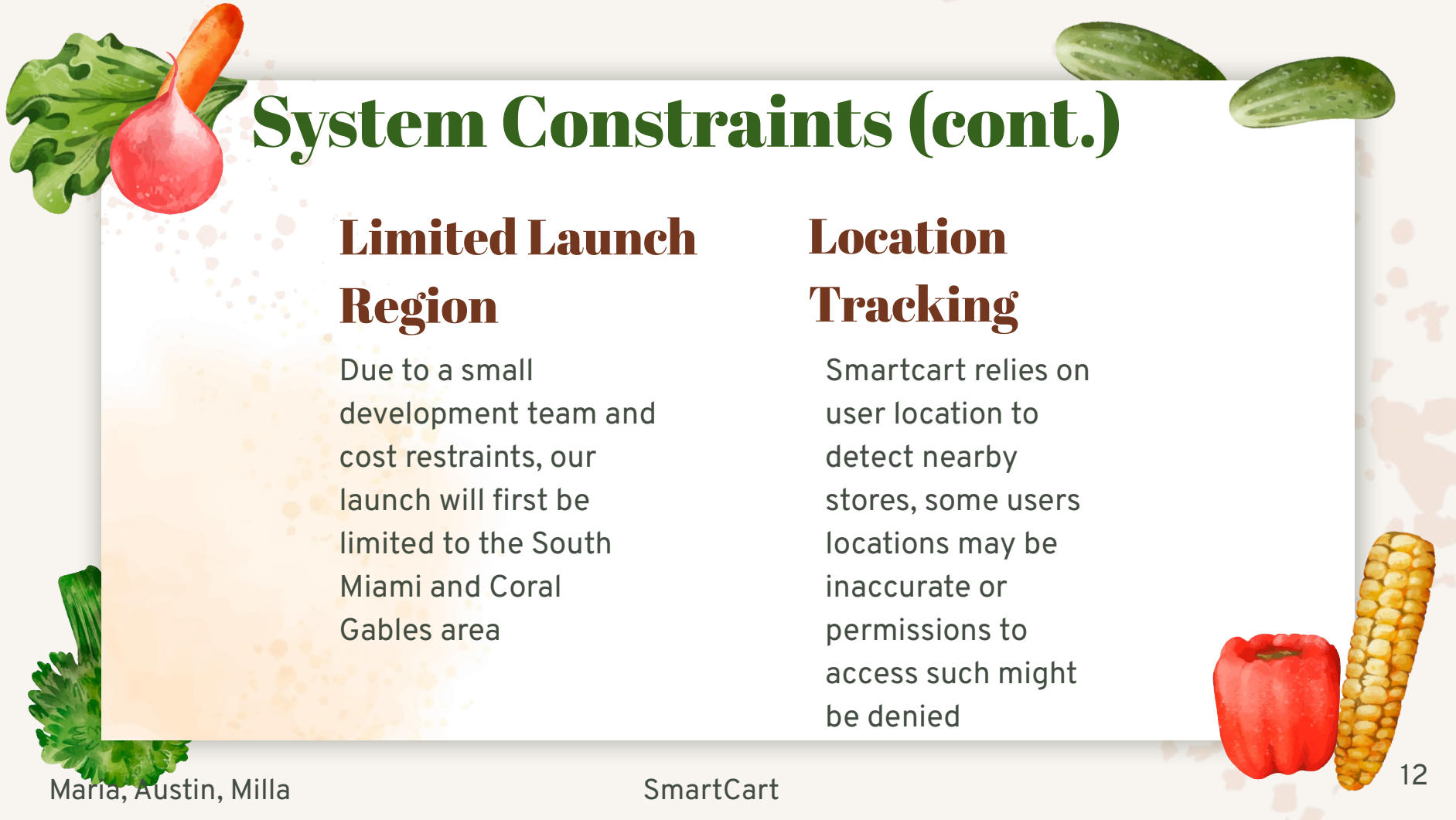
SmartCart is designed conceptually as a web-based platform accessible from standard browsers. No special hardware is required beyond a normal student laptop.

Internet Required

The system requires internet access to retrieve and display grocery pricing information.

Data Constraints

The app can only display prices and stores accessed through APIs, web scraping, some stores might not display such information online



System Constraints (cont.)

Limited Launch Region

Due to a small development team and cost restraints, our launch will first be limited to the South Miami and Coral Gables area

Location Tracking

Smartcart relies on user location to detect nearby stores, some users locations may be inaccurate or permissions to access such might be denied



05

Evolution

Maria, Austin, Milla

SmartCart

13





Evolutionary Requirements



User Account Management

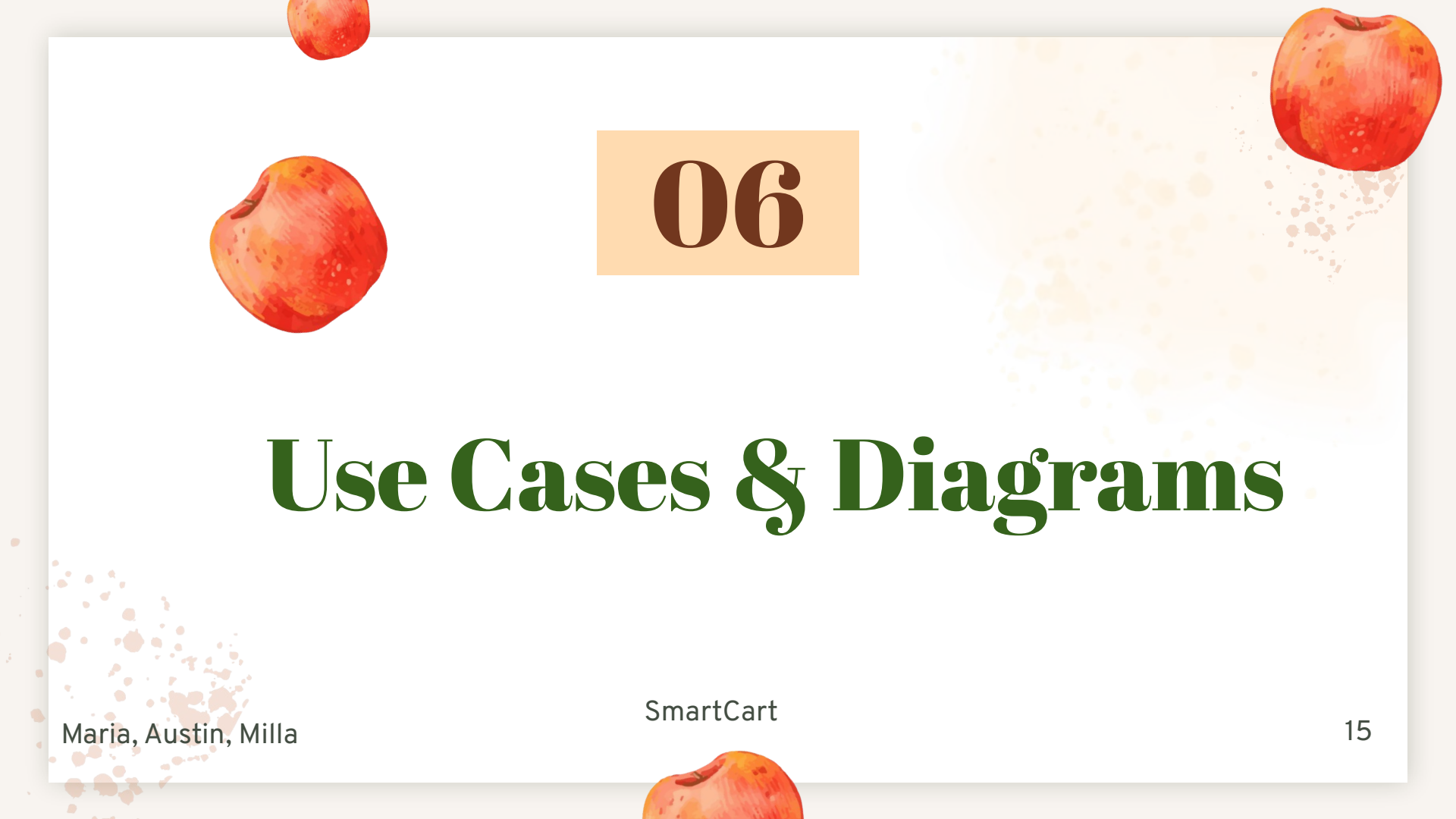
Future versions may allow users to create accounts to save grocery lists and preferences between sessions. Requires secure authentication.
(FR-4.1.1, Priority 3)

Location-Based Filtering

Future versions may allow users to set a location or travel radius so only nearby stores appear in comparisons. (FR-4.1.2, Priority 3)

Scalability & Security

Support larger datasets and more stores without performance issues. If accounts are added, personal data must be protected with secure authentication.
(NFR-4.2.1 & 4.2.2)

The slide features a decorative background with several red apples and orange splatters. One apple is at the top left, another at the top right, and a third at the bottom center. Orange splatters are scattered across the slide, with a large cluster on the right side and smaller ones on the left and bottom.

06

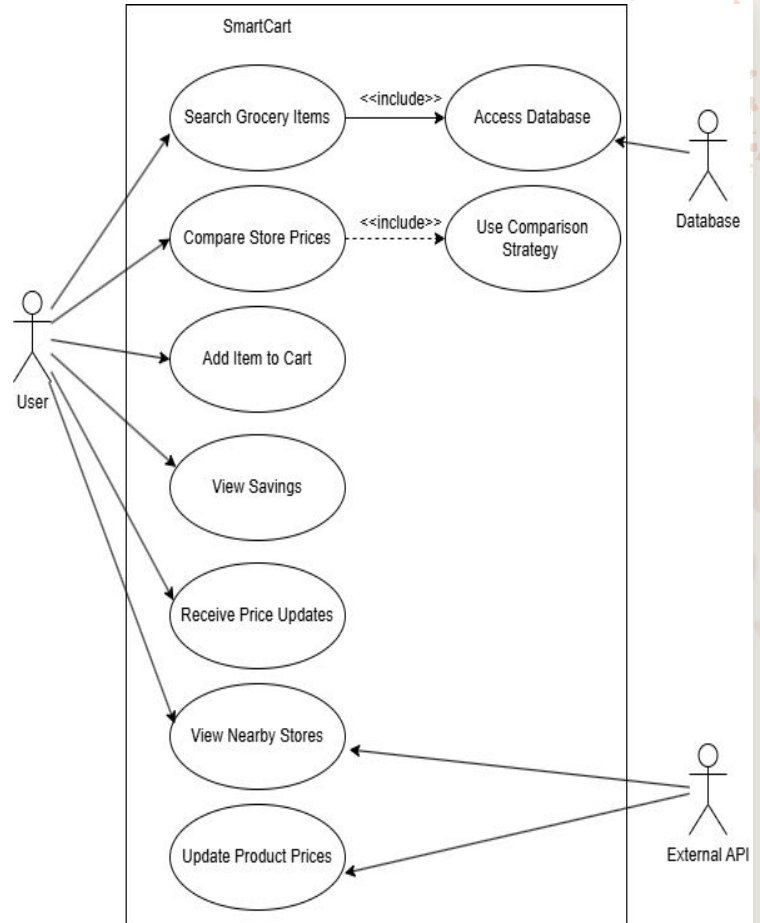
Use Cases & Diagrams

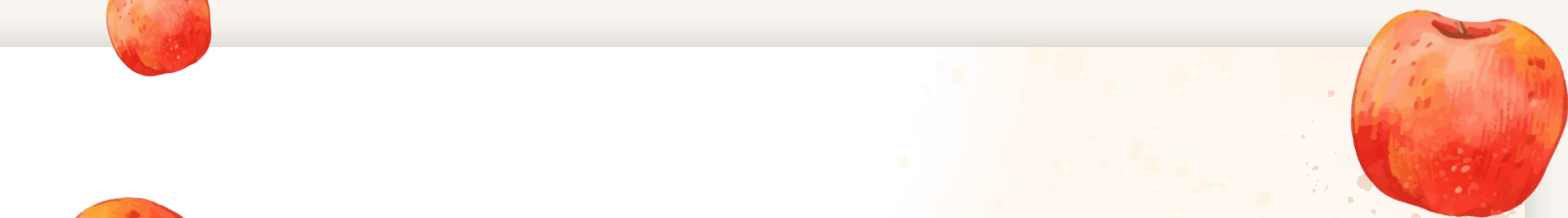


Use Case Diagram — SmartCart

This figure illustrates the core interactions between a user and the SmartCart system. Representing the primary functionality.

- The user can search for grocery items
- Allows users to compare prices across stores,
- The system calculates total cart costs
- Gives users a view of their potential savings





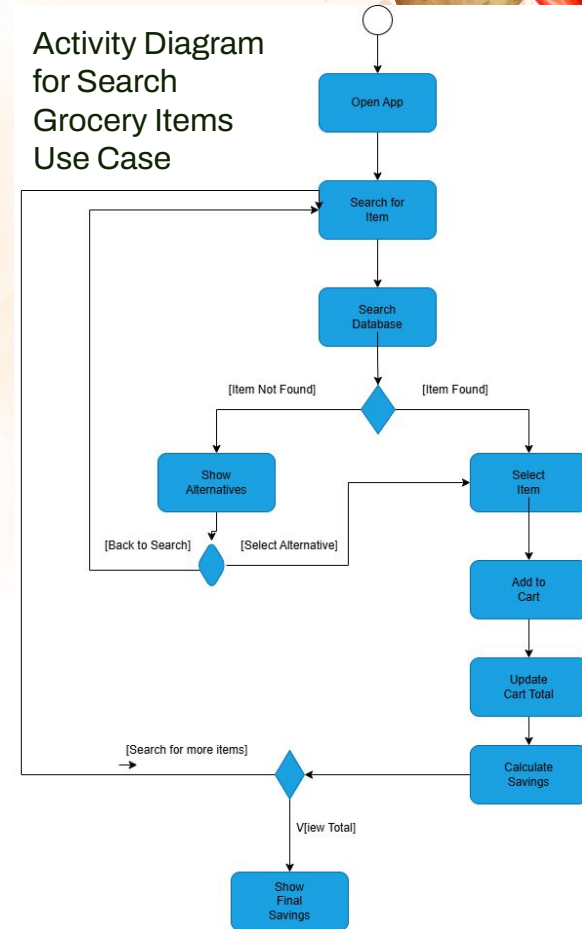
07

System Model

High-Level Model View and Frameworks

- Users access the platform through a web browser
- Frontend handles user interface and input
- Backend processes, requests, and performs calculations
- External APIs such as FoodSpark provide store locations and pricing data
- Database stores product, pricing, and user data

Activity Diagram for Search Grocery Items Use Case



System Frameworks

Frontend



Backend



Database





08

System Architecture



Maria, Austin, Milla

SmartCart

20

SmartCarts Layered Architecture

- Presentation layer manages UI and user interactions
- Application layer handles business logic and price comparisons
- Data layer stores structured grocery and user data
- APIs connect to external grocery store systems
- Architecture supports scalability and maintainability

Presentation Layer

SmartCart Web Application React.js Logic

Application Layer

Item Search Cart Cost Calculations Comparison Strategies Savings Highlights
Price Update Broadcasting Location Calculations Cart Update Logic

Data Layer

Authentication User Data CSV Tables Web Server Logs

API & Web Tools

External Price APIs (FoodSpark)

Direct Vendor APIs

The slide features a white background with decorative elements. In the top-left corner, there is a small red apple. In the top-right corner, there is a larger red apple. In the bottom-left corner, there are several small orange and red splatters. In the bottom-center, there is a small red apple. The number '09' is displayed in a large, bold, dark brown font inside an orange rectangular box.

09

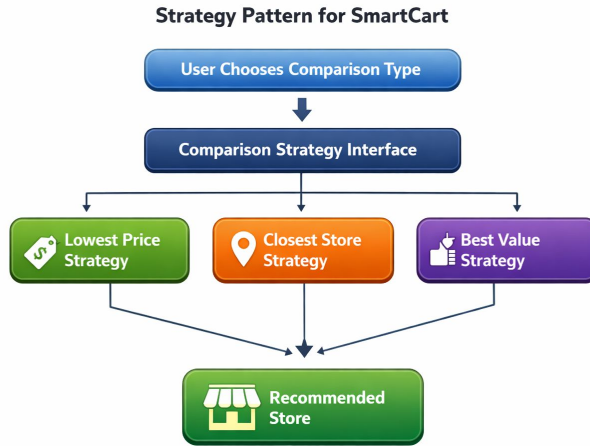
Design Patterns

Maria, Austin, Milla

SmartCart

22

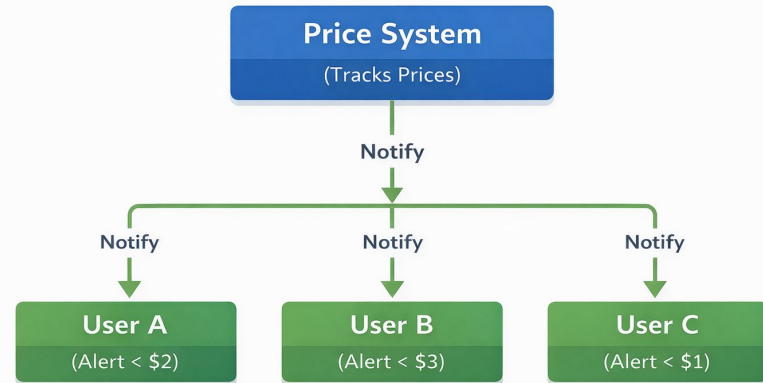
Strategy Pattern — Item Comparison



- Makes it easy to switch between different comparison methods without changing the rest of the system
- Supports multiple strategies such as lowest total price, closest store, or best overall value
- Keeps the comparison logic separate from the user interface and database logic
- Reduces duplicated code by keeping each comparison method in its own class or module

Observer Pattern — Price Updates

- The price system acts as the **subject**
- Users act as **observers**
- When a price changes, all subscribed views are automatically updated
- Eliminates need for constant manual checking

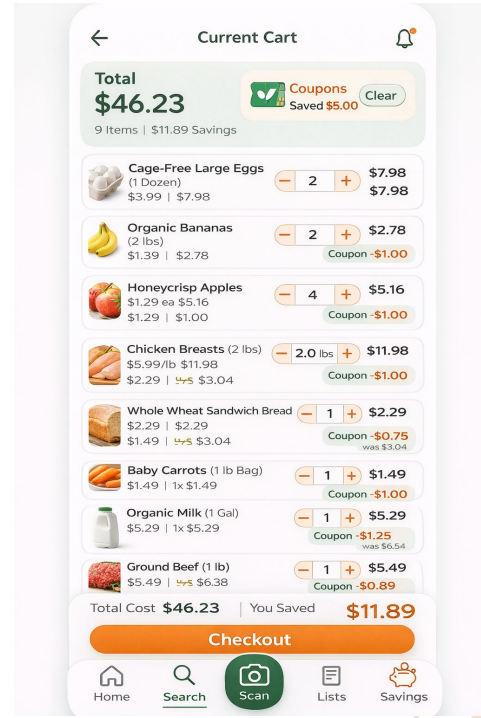
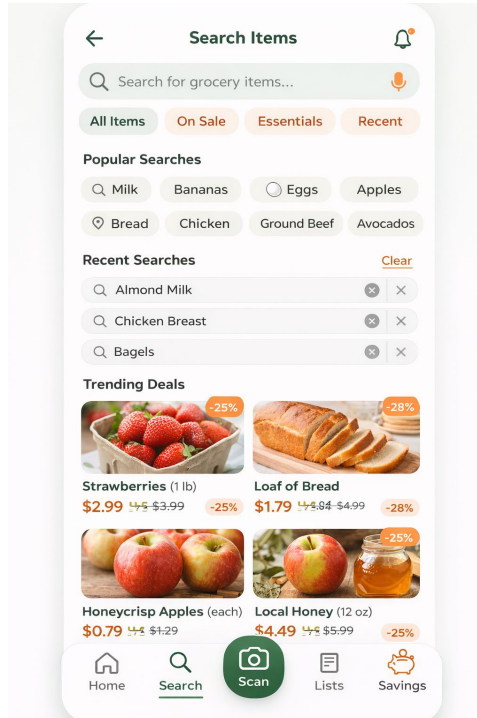
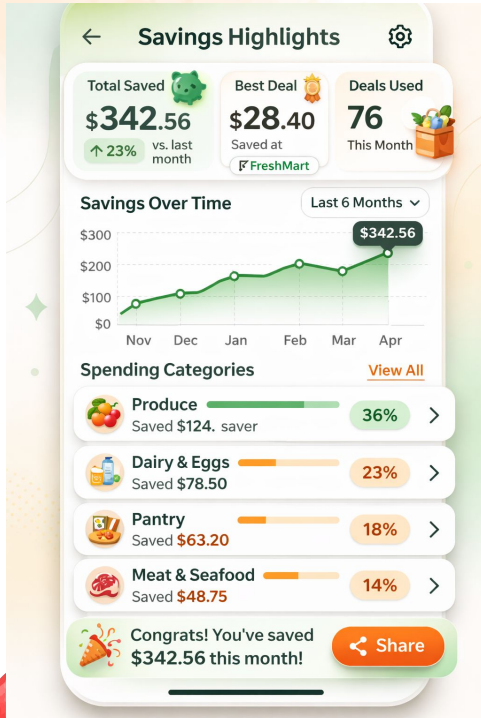




10

Mockup Interface

Mockup Interface — SmartCart





11

Conclusion

Conclusion

SmartCart Recap

- Make Shopping Easier
- Find all the best deals
- View your savings
- Access it all from your browser

Design Choices

- Layered Architecture simplifies system
- Observer pattern for price updates
- Strategy pattern for deal selections



Thanks!

Do you have any questions?

Vote for TEAM 5

DynaFlip.com/p1105

